AI ASSISTED CODING
ASSIGNMENT - 14.4
2303A510A0 - VISHNU VARDHAN
Batch - 14

Lab 14: Web Frontend Development – AI-Assisted HTML/CSS/JS Generation

Lab Objectives

- Design functional, visually appealing web applications using HTML, CSS, and JavaScript with AI assistance.
- Apply responsive, accessible, and interactive design principles.
- Create practical UI components for real-world web applications.
- Use AI to optimize layout, UX, and performance.

Lab Outcomes

- Applying AI assistance to frontend development tasks
- Understanding how HTML, CSS, and JavaScript work together
- Improving responsiveness, interactivity, and usability
- Evaluating and refining AI-generated frontend code

Task 1: Responsive Homepage Layout for a Startup Website

Scenario

You are designing a basic homepage for a startup company that needs

to look good on desktops, tablets, and mobile devices.

Task Description

- Create a web page structure using HTML that includes:
    - A header with navigation links

- A main content section
- A footer with basic information

• Use AI assistance to generate responsive CSS using

media queries.

• Ensure the layout adjusts smoothly across different screen

sizes. Expected Output

• A responsive web page where:

- Header, content, and footer are clearly separated
- Layout adapts correctly to mobile, tablet, and desktop views
- Code is clean and well-structured

Step 1

Open a **code editor**.

Step 2

Create a project folder -STARTUP-HOMEPAGE

Step 3

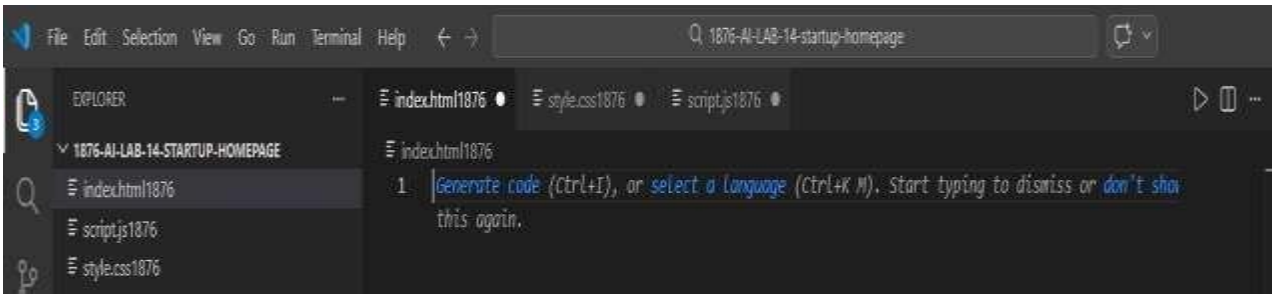
Inside the folder create three files:

index.html

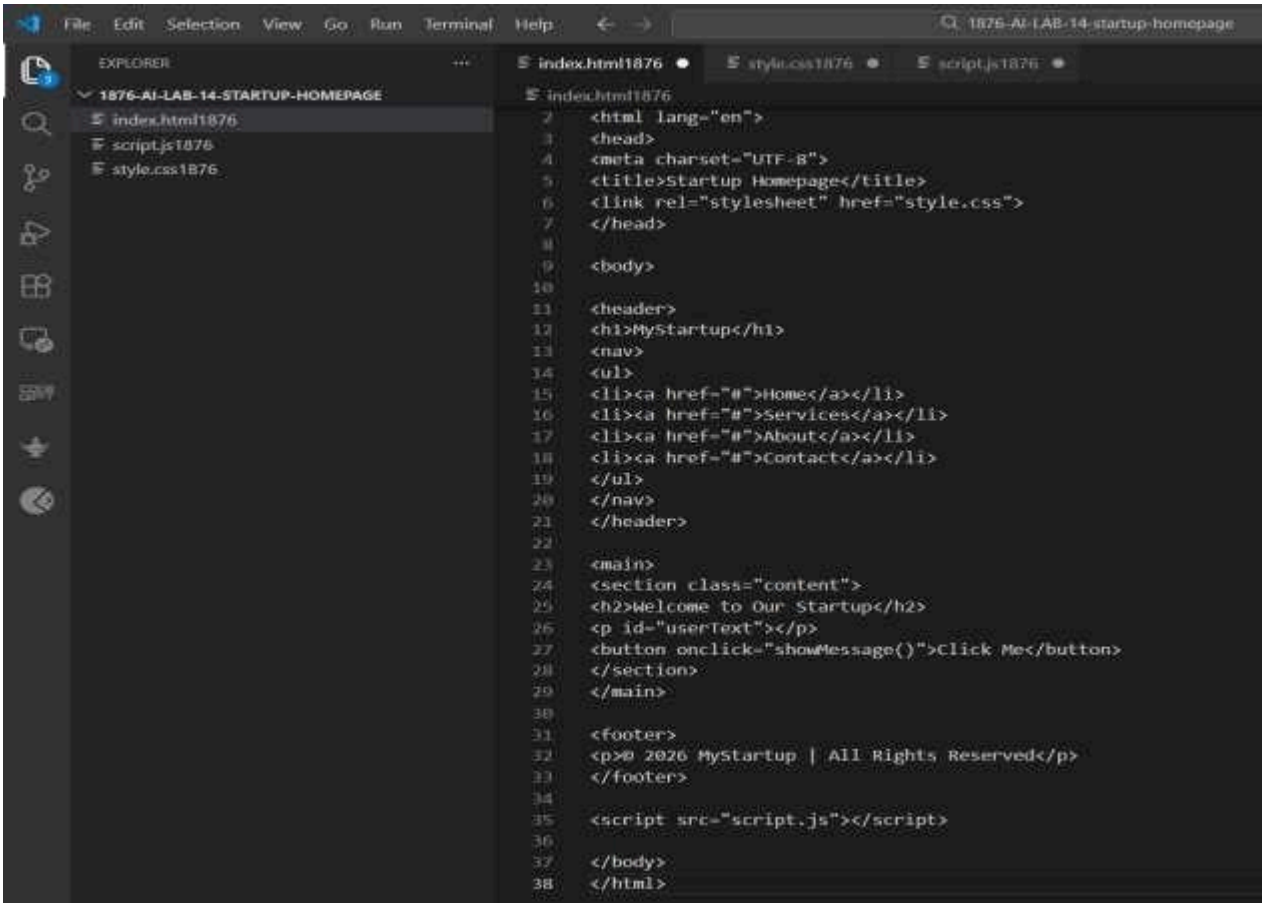
style.css

script.js

Screenshot



index.html



Output

MyStartup

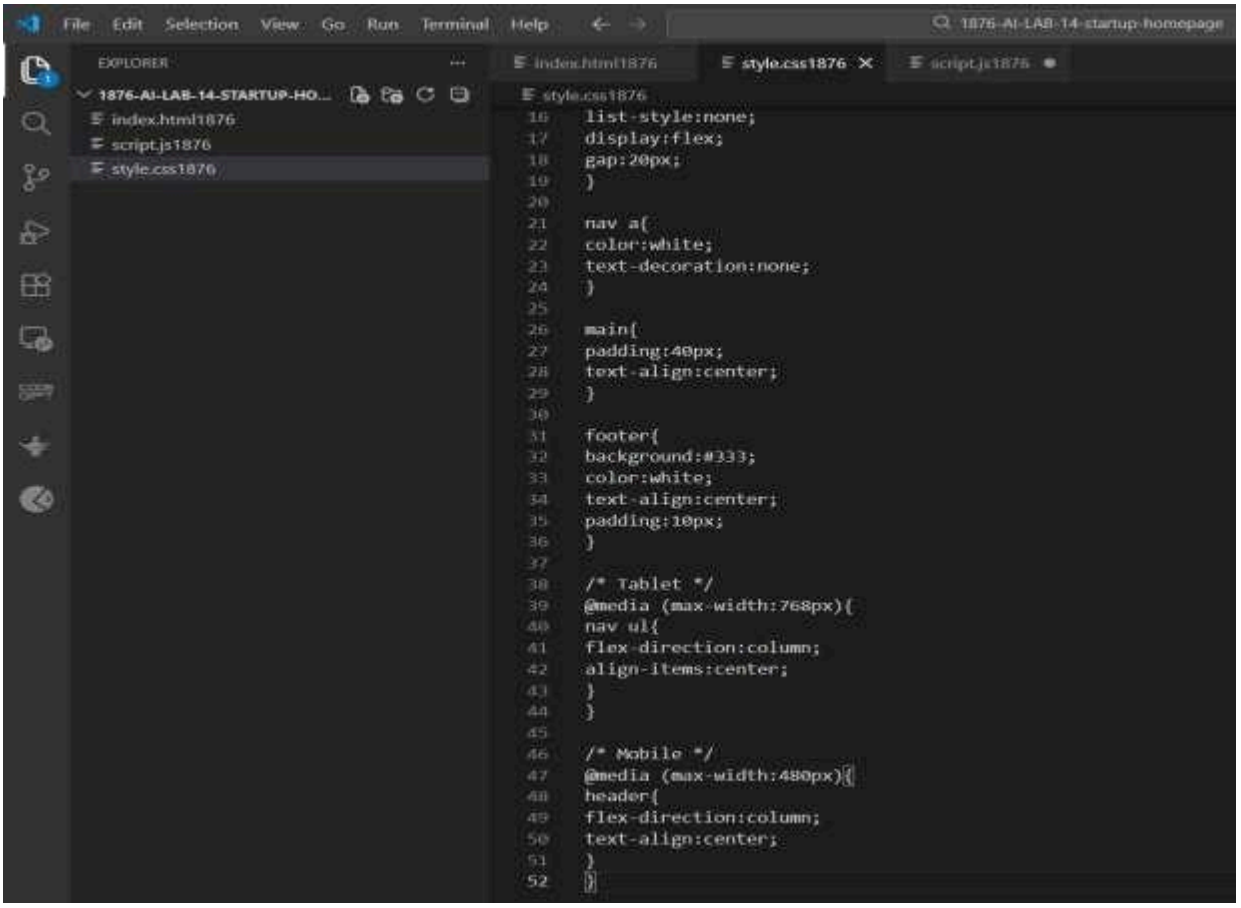
- Home
- Services
- About
- Contact

Welcome to Our Startup

Click Me

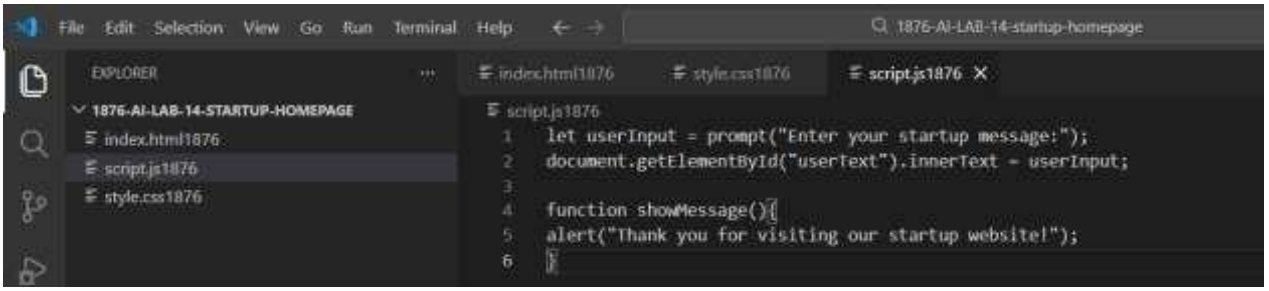
© 2026 MyStartup | All Rights Reserved

style.css



Output

script.js



Output

Observation

The webpage successfully displays a structured layout with header, content section, and footer. The responsive CSS ensures the navigation menu adjusts properly for different screen sizes. The JavaScript allows user input to be displayed dynamically on the page.

Explanation

HTML is used to create the basic webpage structure. CSS styles the webpage and media queries allow the layout to adapt to different devices. JavaScript adds simple interactivity by taking input from the user and displaying it on the page.

Justification

A responsive design is essential for modern websites because users access websites from various devices such as smartphones, tablets, and desktops. Using AI-assisted code generation helps developers quickly create optimized and responsive layouts.

Conclusion

In this lab, a responsive startup homepage was successfully developed using HTML, CSS, and JavaScript. The webpage structure, styling, and interactivity demonstrate how frontend technologies work together to build modern and user-friendly web applications.

Task 2: Interactive Call-to-Action Button

Scenario

A marketing page requires a call-to-action button that responds when users interact with it.

Task Description

• Add a button to a web page using HTML.

• Use AI to generate JavaScript that responds to the button click by:

- Displaying a message or alert

• Ask AI to add meaningful comments explaining the

JavaScript logic.

Expected Output

• A web page where:

- Clicking the button triggers a visible response (alert

or message)

- JavaScript code is readable and commented
- No page reload occurs

Step 1

Open a **code editor**.

Step 2

Create a project folder – CTA-BUTTON

Step 3

Inside the folder create three files:

index.html

style.css

script.js

Step 4

Write HTML code to add a **Call-to-Action button**.

Step 5

Use CSS to style the button.

Step 6

Use JavaScript to detect the **button click event**.

Step 7

Display a **message or alert when the button is clicked**.

Step 8

Add **comments in JavaScript** explaining the logic.

Step 9

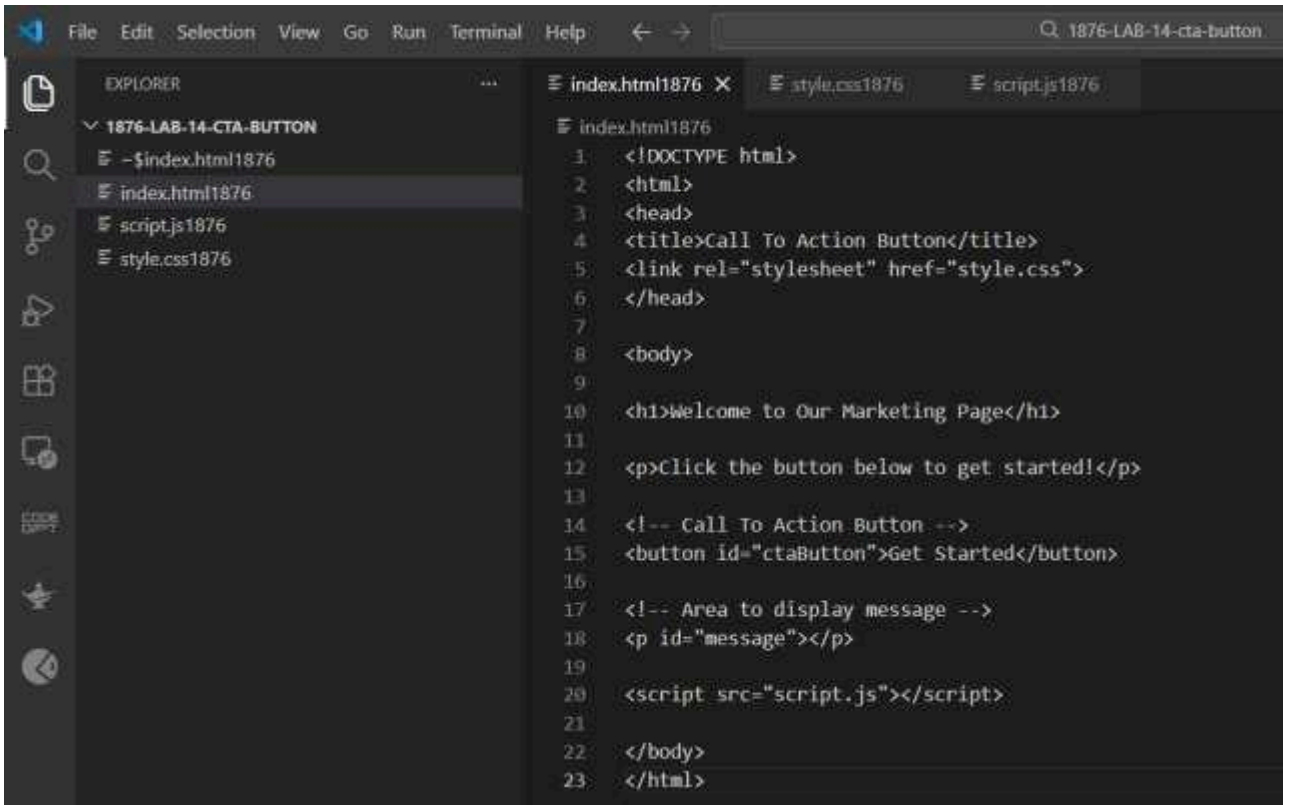
Save all files.

Step 10

Open **index.html in a browser** to see the output.

Screenshot

index.html



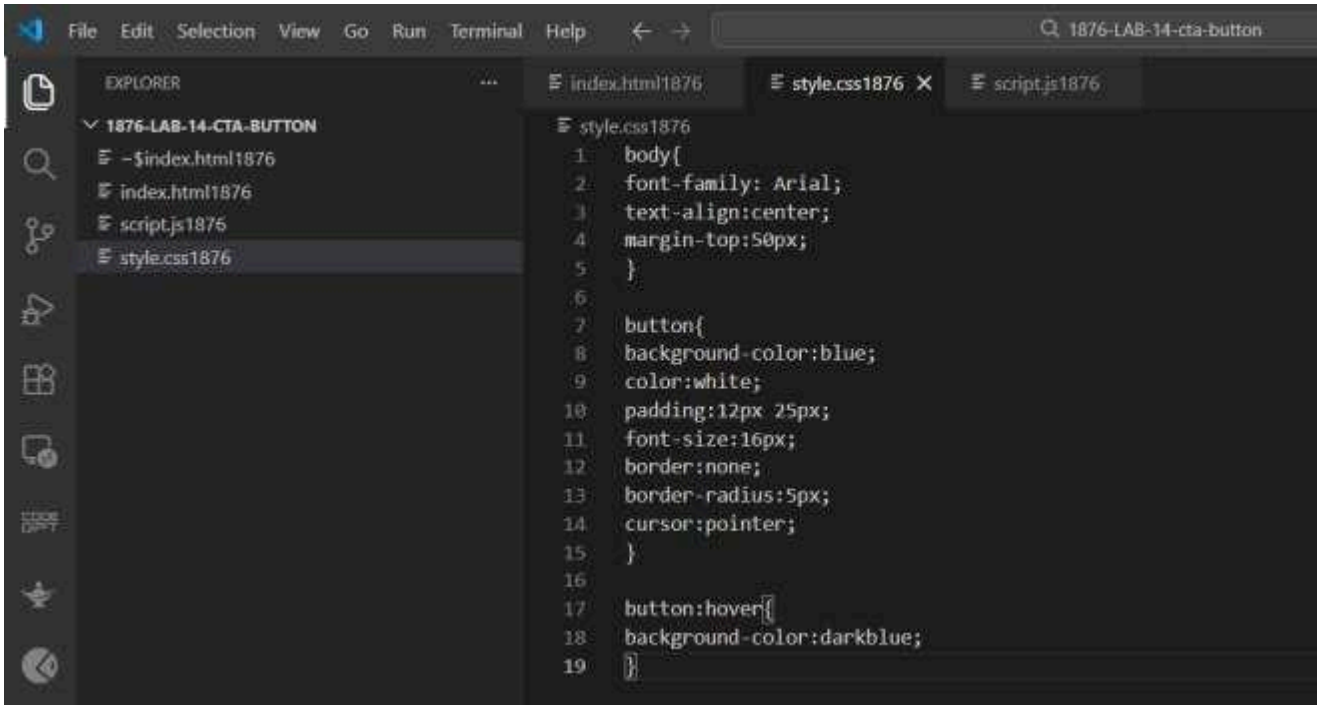
Output

Welcome to Our Marketing Page

Click the button below to get started!

Get Started

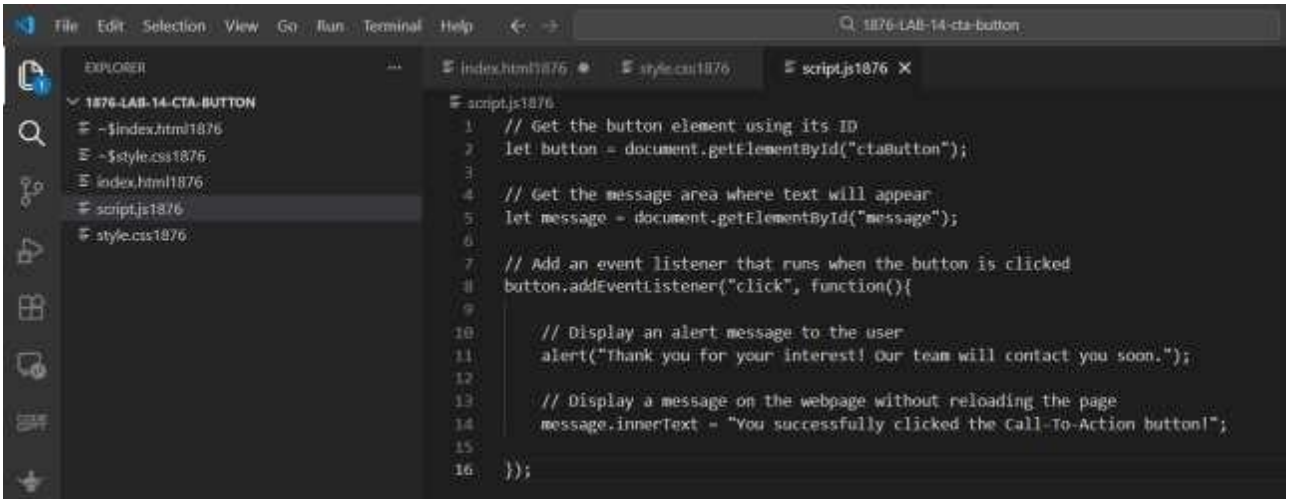
style.css



Output

```
body{
font-family: Arial;
text-align:center;
margin-top:50px;
}

button{
background-color:blue;
color:white;
padding:12px 25px;
font-size:16px;
border:none;
border-radius:5px;
cursor:pointer;
}

button:hover{
background-color:darkblue;
}
```

script.js



Output

```
// Get the button element using its ID
let button = document.getElementById("ctaButton");

// Get the message area where text will appear
let message = document.getElementById("message");

// Add an event listener that runs when the button is clicked
button.addEventListener("click", function(){

    // Display an alert message to the user
    alert("Thank you for your interest! Our team will contact you
soon.");

    // Display a message on the webpage without reloading the page
    message.innerText = "You successfully clicked the Call-To-Action
button!";

});
```

Observation

The Call-to-Action button successfully responds when clicked. JavaScript detects the click event and displays an alert message and a text message on the page without refreshing the webpage.

Explanation

HTML creates the webpage structure and the button. CSS is used to style the button for better appearance. JavaScript handles the button click event using addEventListener() and shows a message to the user dynamically.

Justification

Interactive buttons improve user engagement on websites. Using JavaScript allows developers to respond to user actions instantly without reloading the page, improving the user experience.

Conclusion

In this task, an interactive Call-to-Action button was successfully implemented using HTML, CSS, and JavaScript. The button responds to user clicks by displaying an alert and a message, demonstrating basic frontend interactivity.

Task 3: Contact Form with Client-Side Validation

Scenario

You are developing a contact form for a company website to collect user queries safely and correctly.

Task Description

• Design a form with fields for:

- Name
- Email
- Message

• Use AI to generate JavaScript validation logic that:

- Prevents empty submissions
- Checks for a valid email format

• Display clear error messages for invalid input.

Expected Output

• A functional form where:

- Invalid input shows inline error messages
- Valid input displays a success message
- Validation happens on the client side using JavaScript

Step 1

Open **Visual Studio Code**.

Step 2

Create a folder:

contact-form

Step 3

Inside the folder create three files:

index.html
style.css
script.js

Step 4

Write HTML code to create a **contact form** with fields:

- Name
- Email
- Message

Step 5

Use CSS to style the form and error messages.

Step 6

Use JavaScript to validate:

- Empty fields
- Email format

Step 7

Display **error messages if input is invalid**.

Step 8

Display **success message if input is valid**.

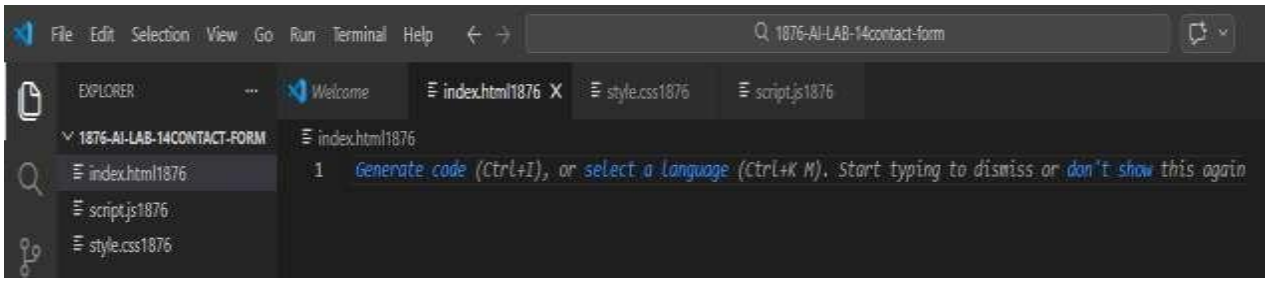
Step 9

Save the files.

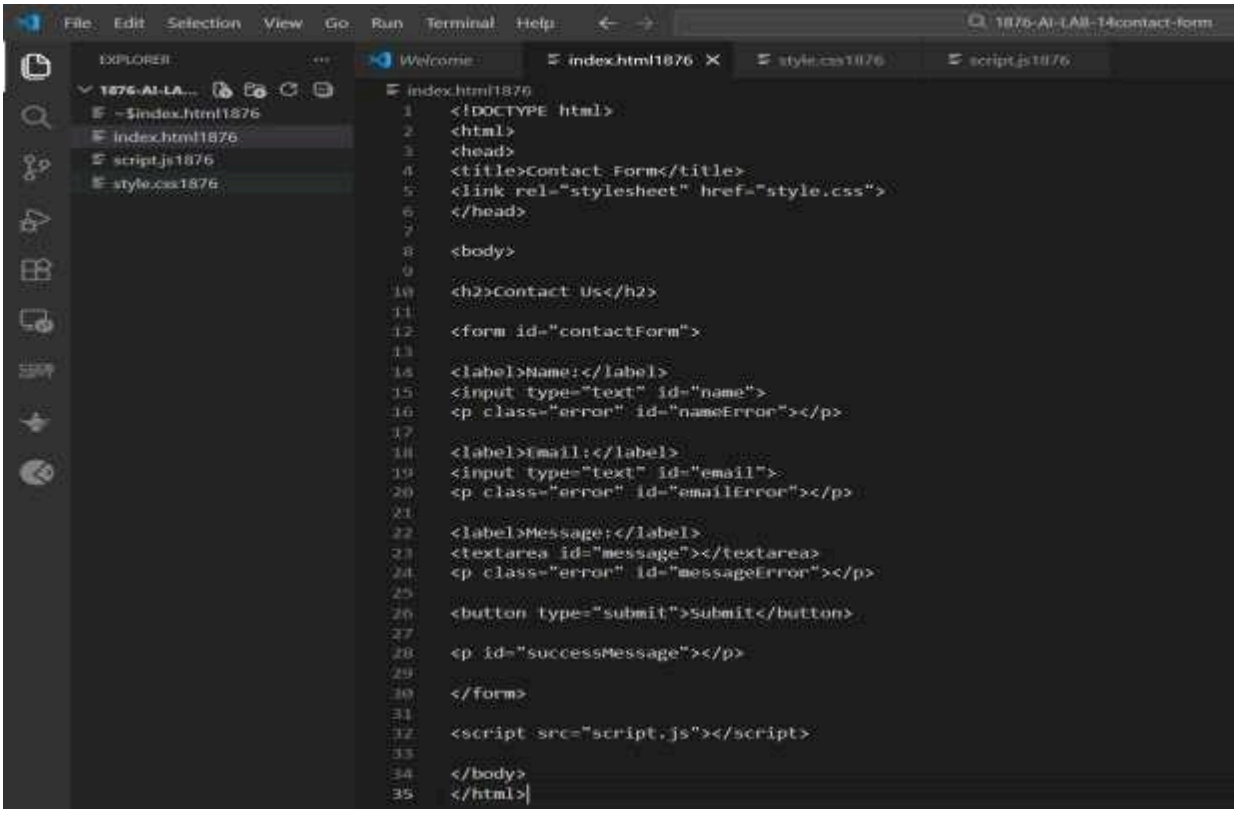
Step 10

Open **index.html in a browser** to test the form.

Screenshot



index.html



Output

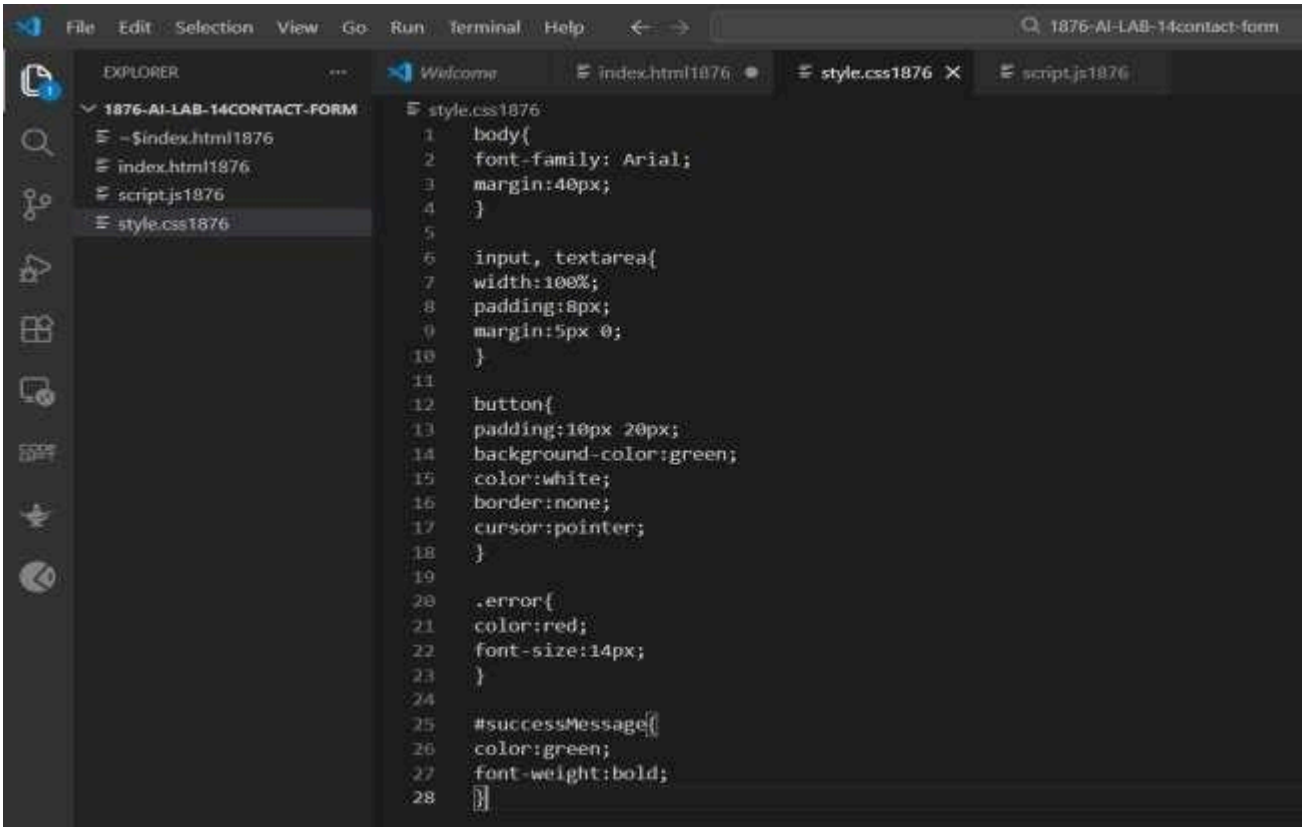
Contact Us

Name:
Email:
Message:
Submit

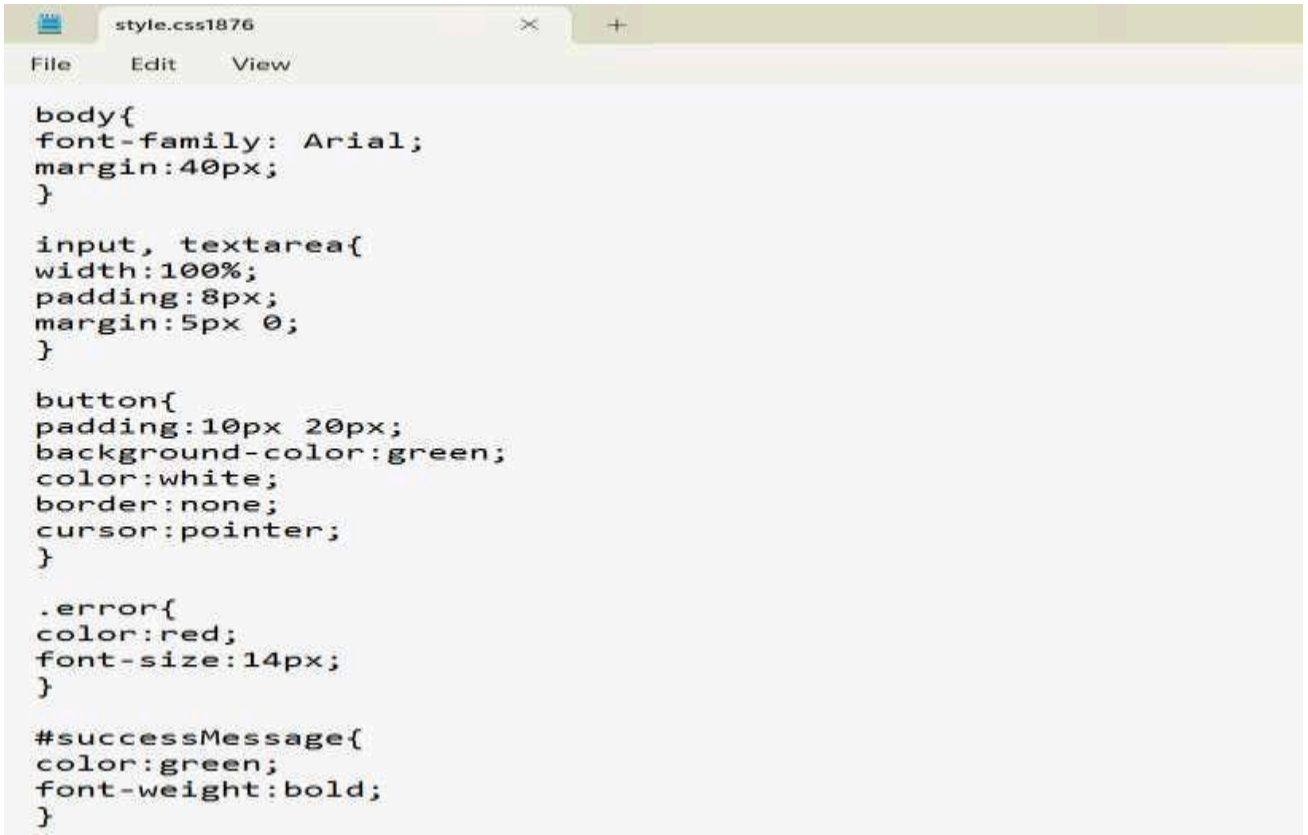
Contact Us

Name: T. Shylasri
Email: shyluthulasi@gmail.com
Message: I would like to know more about your services.
Submit

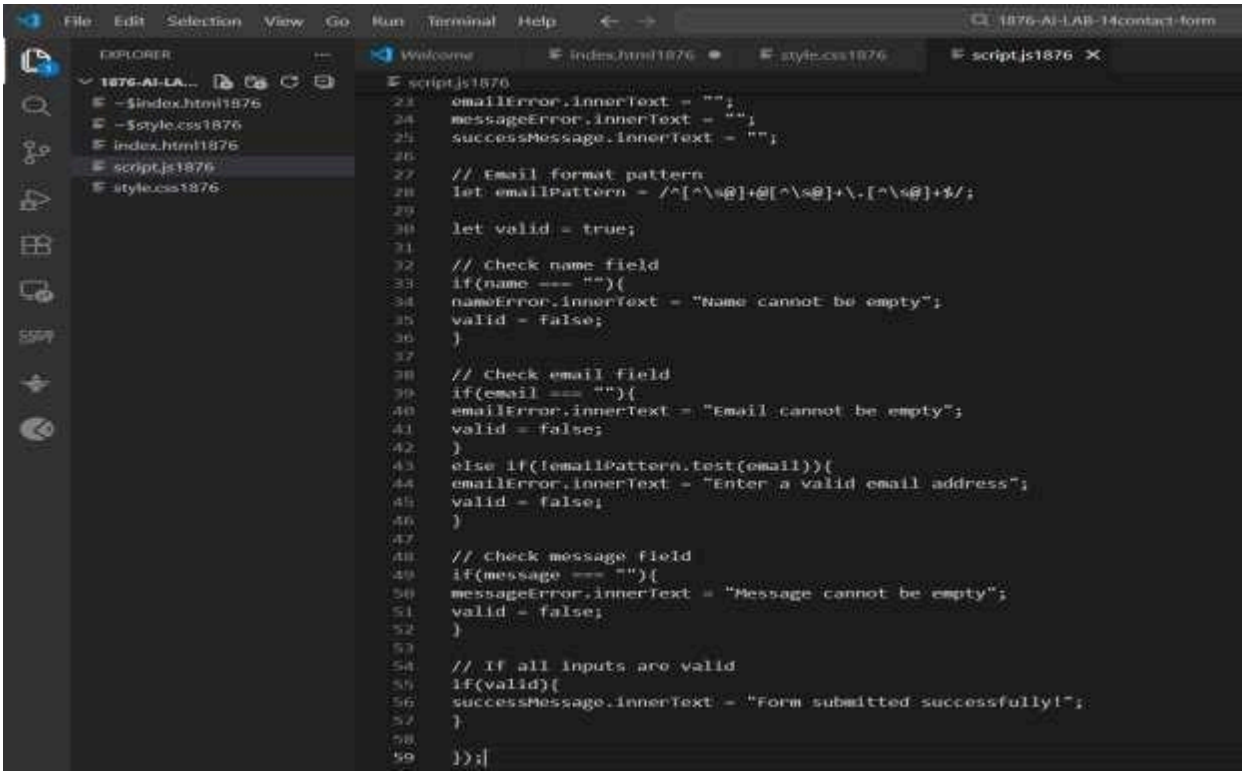
style.css



Output

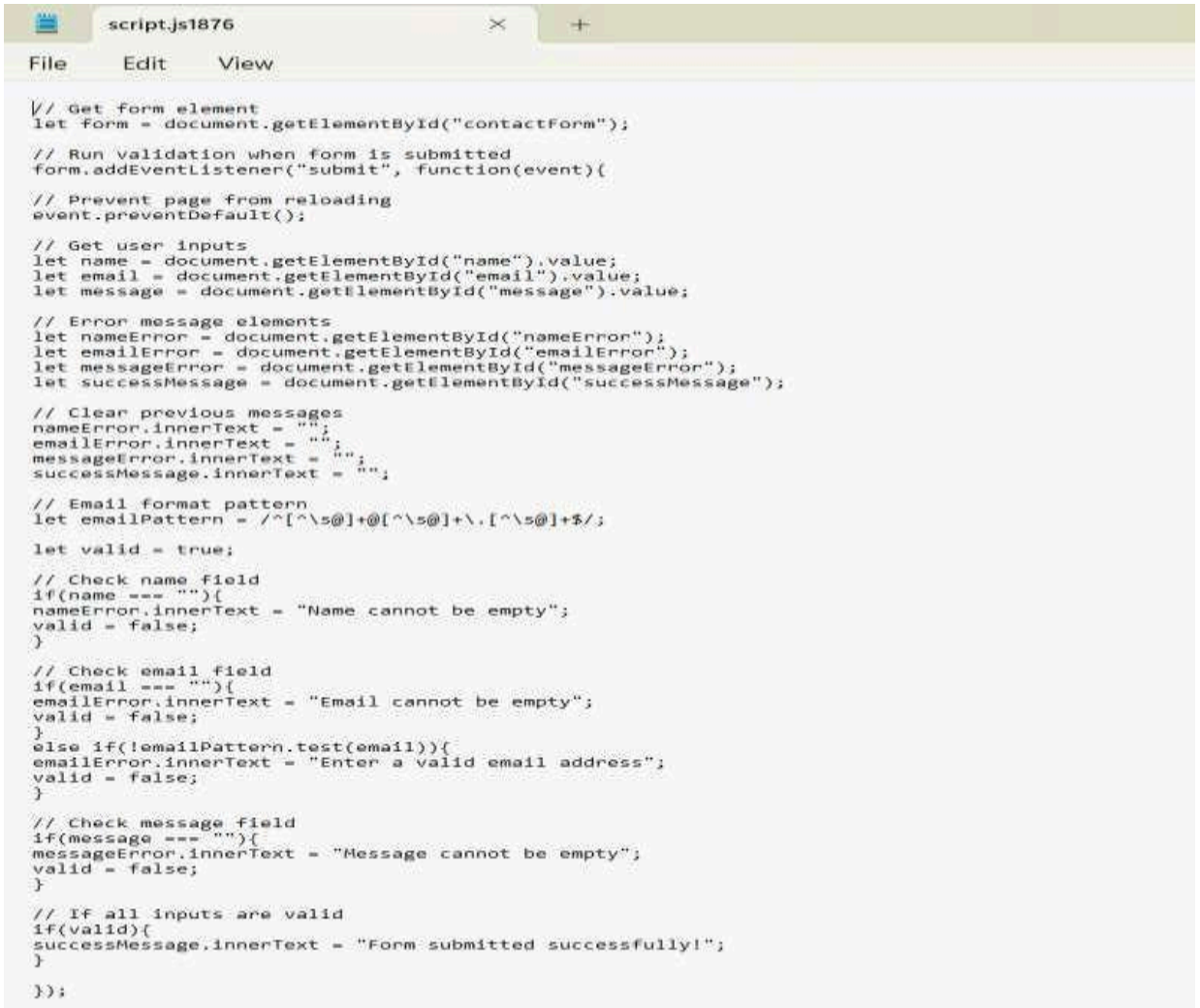


script.js



```
    emailError.innerText = "";
    messageError.innerText = "";
    successMessage.innerText = "";

    // Email format pattern
    let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    let valid = true;

    // Check name field
    if(name === ""){
    nameError.innerText = "Name cannot be empty";
    valid = false;
    }

    // Check email field
    if(email === ""){
    emailError.innerText = "Email cannot be empty";
    valid = false;
    }
    else if(!emailPattern.test(email)){
    emailError.innerText = "Enter a valid email address";
    valid = false;
    }

    // Check message field
    if(message === ""){
    messageError.innerText = "Message cannot be empty";
    valid = false;
    }

    // If all inputs are valid
    if(valid){
    successMessage.innerText = "Form submitted successfully!";
    }

    });
```

Output



```
// Get form element
let form = document.getElementById("contactForm");

// Run validation when form is submitted
form.addEventListener("submit", function(event){

// Prevent page from reloading
event.preventDefault();

// Get user inputs
let name = document.getElementById("name").value;
let email = document.getElementById("email").value;
let message = document.getElementById("message").value;

// Error message elements
let nameError = document.getElementById("nameError");
let emailError = document.getElementById("emailError");
let messageError = document.getElementById("messageError");
let successMessage = document.getElementById("successMessage");

// Clear previous messages
nameError.innerText = "";
emailError.innerText = "";
messageError.innerText = "";
successMessage.innerText = "";

// Email format pattern
let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

let valid = true;

// Check name field
if(name === ""){
nameError.innerText = "Name cannot be empty";
valid = false;
}

// Check email field
if(email === ""){
emailError.innerText = "Email cannot be empty";
valid = false;
}
else if(!emailPattern.test(email)){
emailError.innerText = "Enter a valid email address";
valid = false;
}

// Check message field
if(message === ""){
messageError.innerText = "Message cannot be empty";
valid = false;
}

// If all inputs are valid
if(valid){
successMessage.innerText = "Form submitted successfully!";
}

});
```

Observation

The contact form correctly validates user input using JavaScript. Empty fields and incorrect email formats produce error messages, while valid inputs display a success message.

Explanation

HTML creates the structure of the contact form. CSS improves the visual appearance and highlights error messages. JavaScript performs client-side validation by checking the input fields and email format before allowing submission.

Justification

Client-side validation improves user experience by detecting errors immediately without sending data to the server. It helps ensure that the form receives valid and complete information.

Conclusion

In this task, a contact form with client-side validation was successfully created using HTML, CSS, and JavaScript. The form checks for empty fields and valid email formats, providing clear feedback to the user.

Task 4: Dynamic List Management for a Product Page

Scenario

A product listing page needs to add or remove items dynamically without refreshing the page.

Task Description

• Create a simple HTML list to display items.

• Use AI to generate JavaScript that:

- Adds a new item when an “Add” button is clicked
- Removes the last item when a “Remove” button is clicked

• Ensure the DOM updates dynamically.

Expected Output

• A web page where:

- Items can be added and removed dynamically
- Changes appear instantly without page reload
- JavaScript logic is clear and maintainable

Step 1

Open **Visual Studio Code**.

Step 2

Create a folder:

product-list

Step 3
Inside the folder create three files:
index.html
style.css
script.js

Step 4

Write HTML code to create a **product list using **.

Step 5

Add two buttons:

- **Add Item**
- **Remove**

Item Step 6

Use CSS to style the list and buttons.

Step 7

Use JavaScript to:

- Add a new item to the list
- Remove the last item from the list

Step 8

Save the files.

Step 9

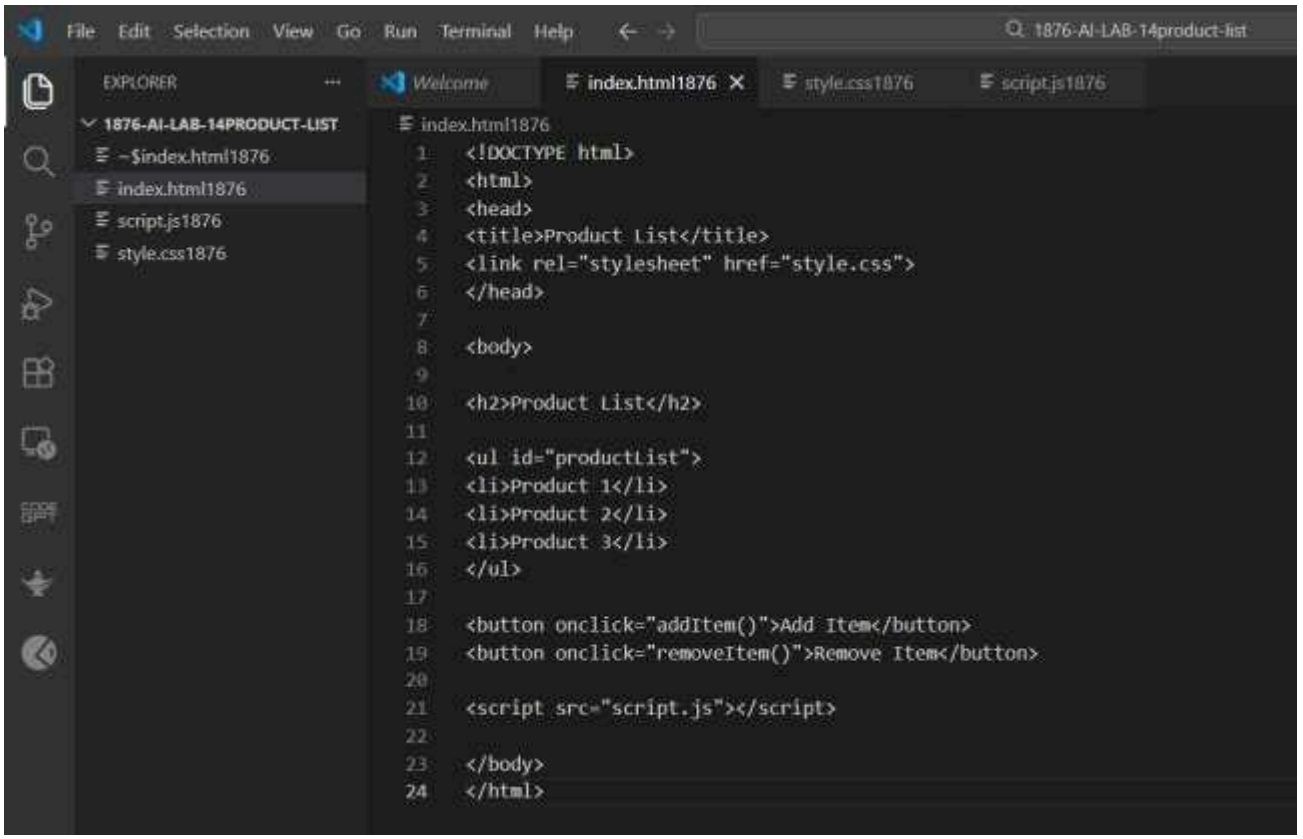
Open **index.html in a browser**.

Step 10

Click the buttons to test the dynamic behavior.

Screenshot

index.html



```
<!DOCTYPE html>
<html>
<head>
<title>Product List</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<h2>Product List</h2>

<ul id="productList">
<li>Product 1</li>
<li>Product 2</li>
<li>Product 3</li>
</ul>

<button onclick="addItem()">Add Item</button>
<button onclick="removeItem()">Remove Item</button>

<script src="script.js"></script>

</body>
</html>
```

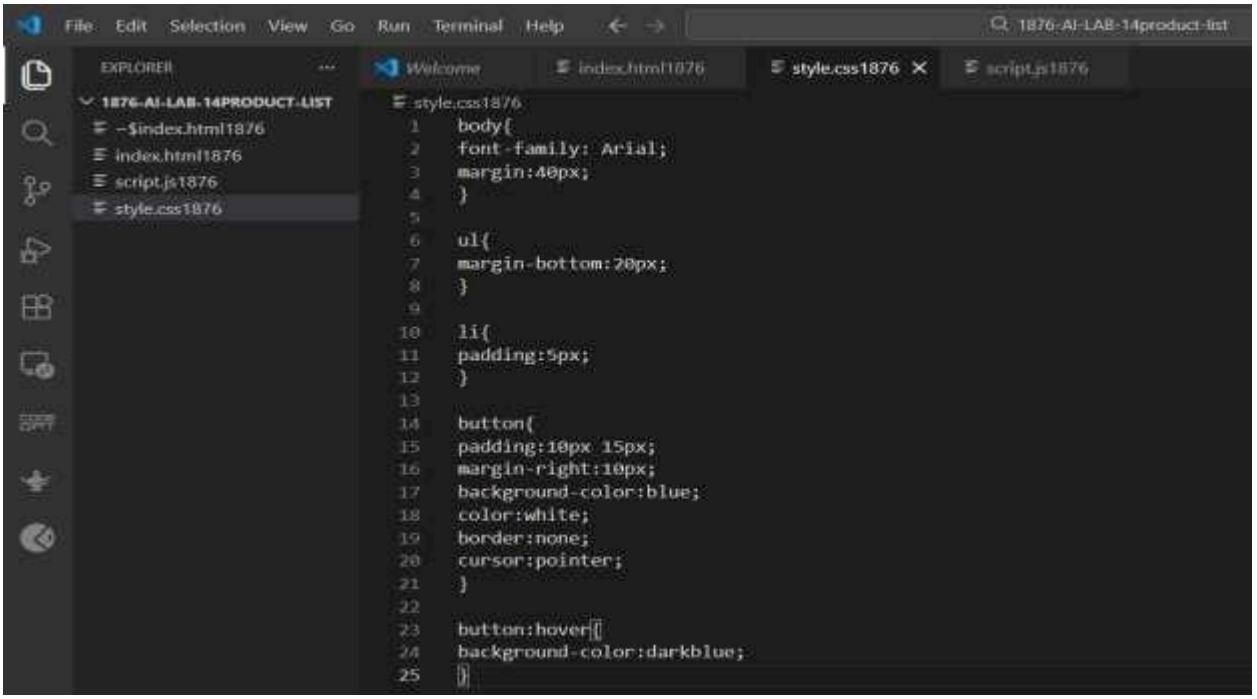
Output

Product List

- Product 1
- Product 2
- Product 3

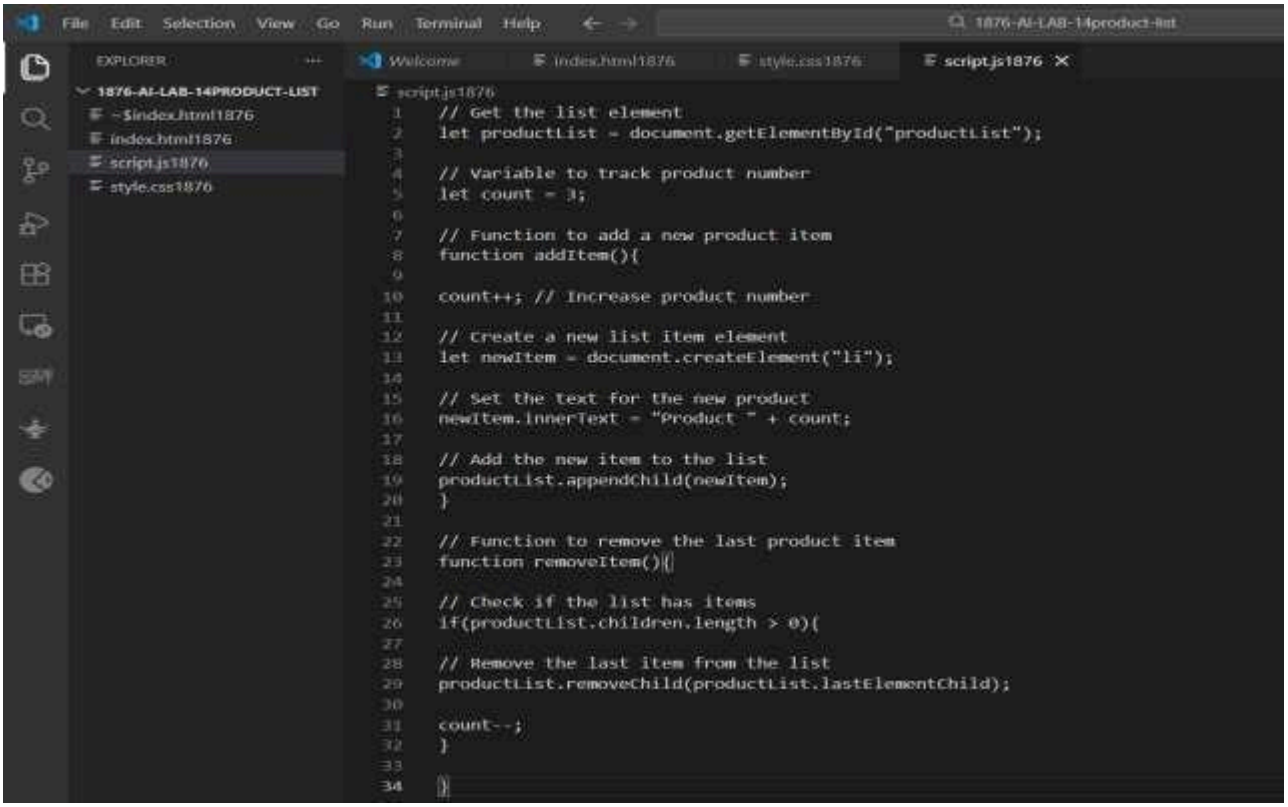
Add Item Remove Item

style.css

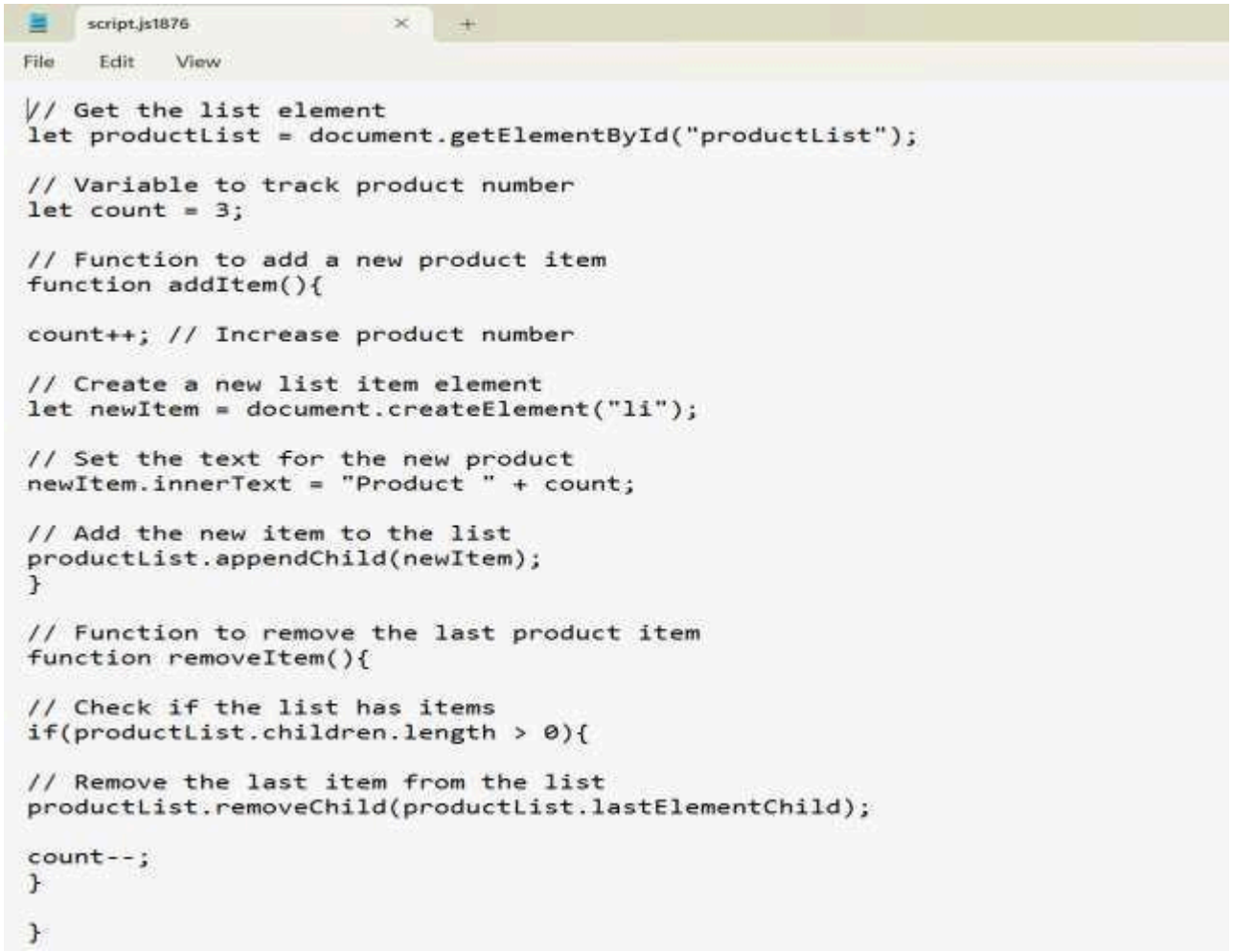


Output

script.js



Output



Observation

The product list updates dynamically when the user clicks the Add or Remove button. New items are appended to the list, and the last item is removed instantly without reloading the page.

Explanation

HTML provides the structure for displaying the list and buttons. CSS styles the list and buttons. JavaScript uses DOM manipulation methods such as createElement(), appendChild(), and removeChild() to dynamically update the list.

Justification

Dynamic DOM manipulation allows web pages to update content instantly without refreshing the page. This improves user experience and is commonly used in modern web applications such as shopping carts and product management systems.

Conclusion

In this task, a dynamic product list was created using HTML, CSS, and JavaScript. Items can be added or removed instantly using buttons, demonstrating the use of DOM manipulation for interactive web pages.

Task 5: Modal Popup for User Notifications

Scenario

You are building a user notification popup for announcements or alerts on a website.

Task Description

• Use AI to help generate:

- HTML structure for a modal popup
- CSS styling with a semi-transparent background overlay
- JavaScript to open and close the modal

• Include multiple ways to close the modal (button or overlay click).

Expected Output

• A web page where:

- Clicking “Open Modal” displays the popup
- Popup overlays the page content clearly
- Modal closes smoothly when instructed

Step 1

Open **Visual Studio Code**.

Step 2

Create a folder:

modal-popup

Step 3
Inside the folder create three files:
index.html
style.css
script.js

Step 4

Write HTML code to create:

- A button **Open Modal**
- A modal popup structure
- A close button inside the modal

Step 5

Use CSS to:

- Create a **semi-transparent background overlay**
- Center the modal on the screen

Step 6

Use JavaScript to:

- Open the modal when clicking **Open Modal**
- Close the modal when clicking **Close button**
- Close the modal when clicking the **overlay area**

Step 7

Save the files.

Step 8

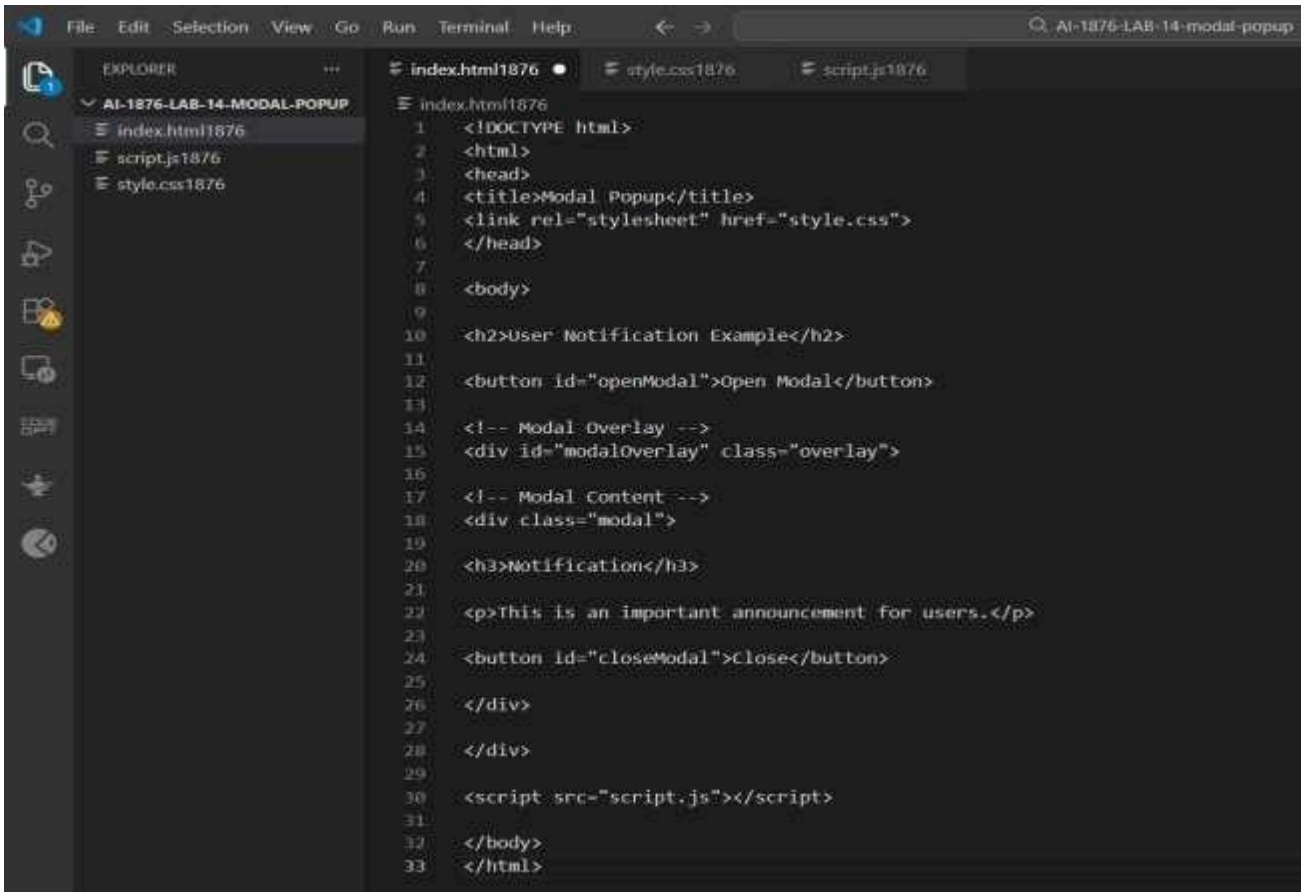
Open **index.html in a browser**.

Step 9

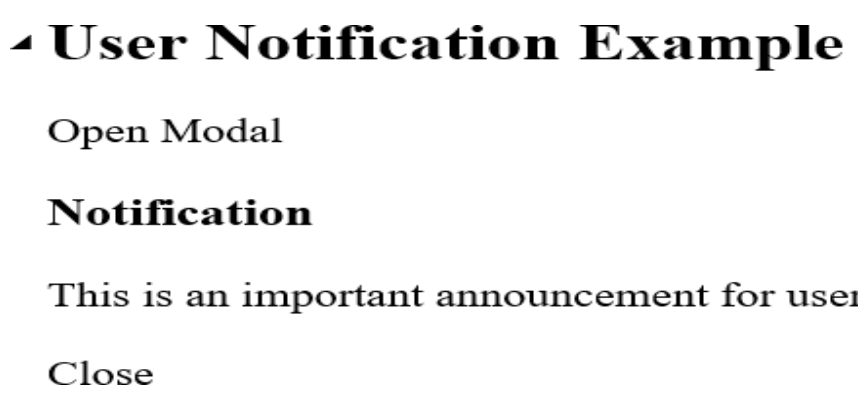
Click the **Open Modal** button to test the popup.

Screenshot

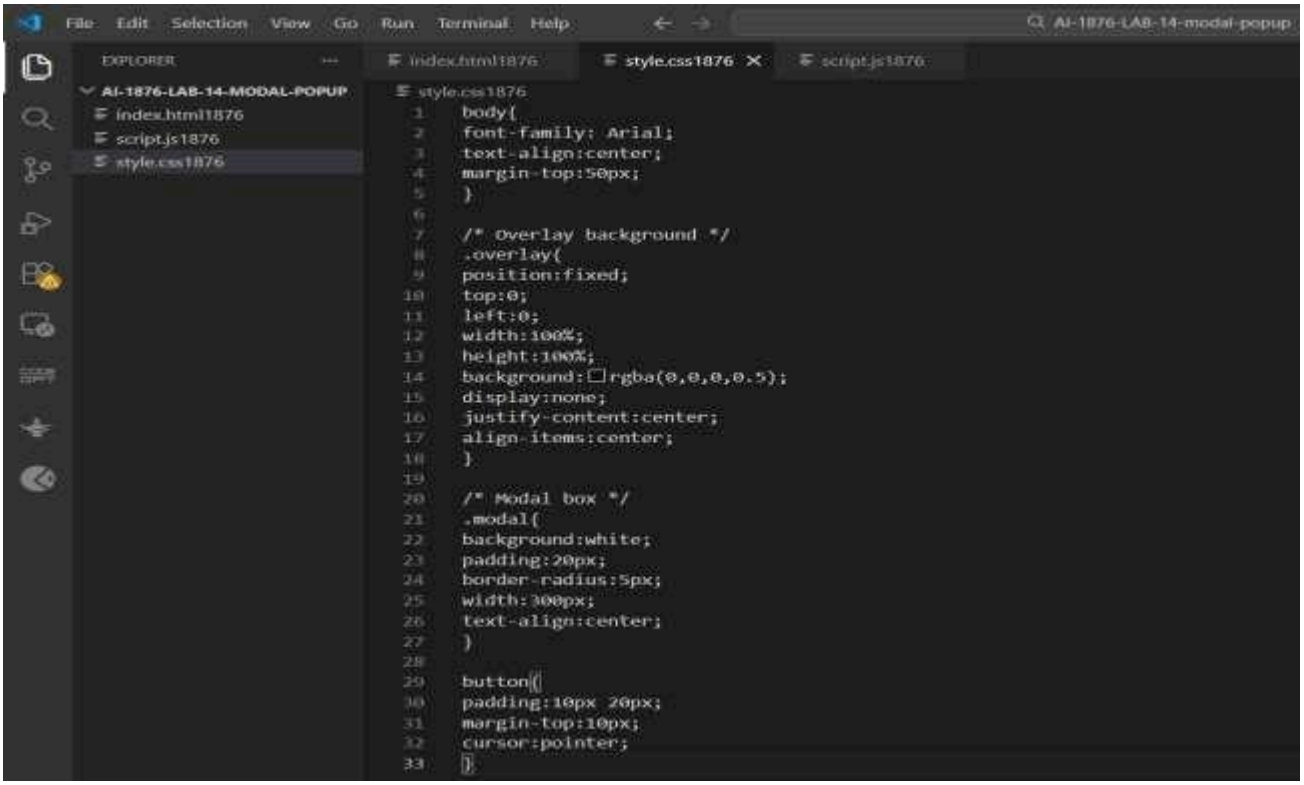
index.html



Output

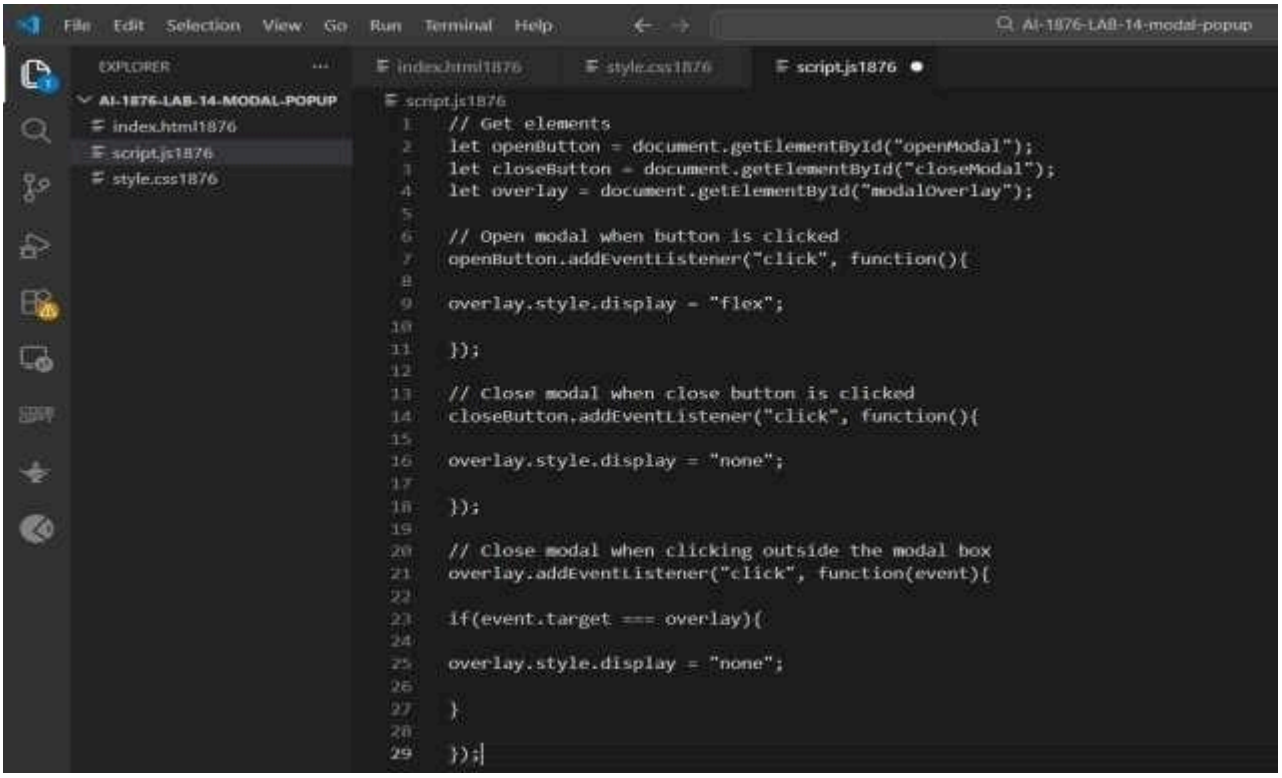


style.css

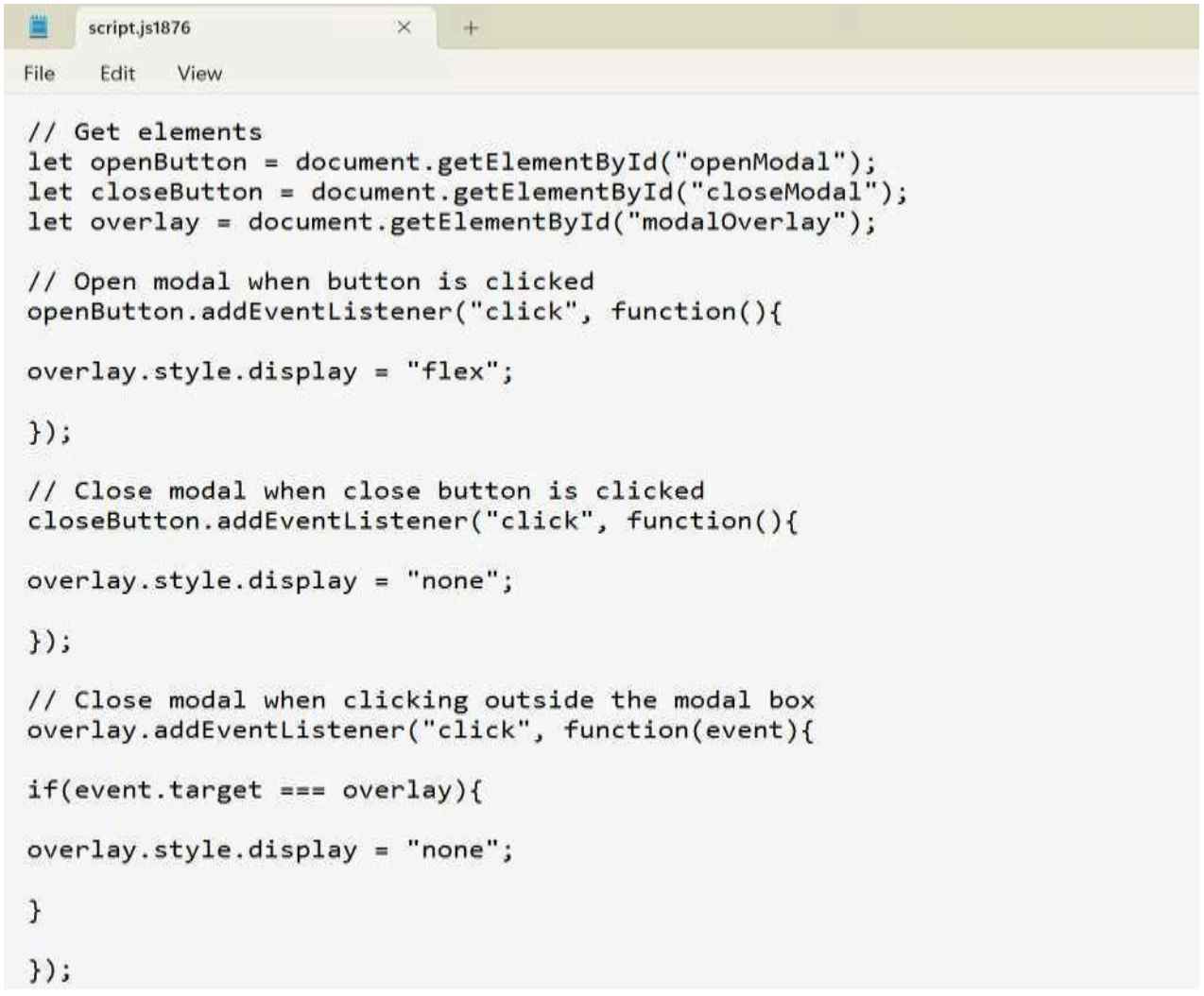


Output

script.js



Output



Observation

The modal popup appears when the Open Modal button is clicked and overlays the webpage with a semi-transparent background. The popup can be closed using multiple methods, providing a smooth user experience.

Explanation

HTML creates the modal structure and button. CSS styles the modal and creates the semi-transparent overlay. JavaScript controls the opening and closing of the modal using event listeners.

Justification

Modal popups are widely used in modern websites to display notifications, alerts, and important messages without navigating to another page. They improve user interaction and keep the user focused on important information.

Conclusion

In this task, a modal popup notification system was successfully created using HTML, CSS, and JavaScript. The popup opens and closes dynamically, demonstrating interactive UI design in frontend development.